Module A — Molécule + Médicament

A

Projet PharmaSémantique · Ontologie Pharmaceutique et Santé · équipe NeoCoders · réalisé par Chemlali Ismail

Objectif. Exploiter la partie **Molécule** et **Médicament** de l'ontologie à travers une application web complète : back-end SPARQL avec inférence OWL, interface graphique, CRUD, et un **assistant qui traduit une question en français en requête SPARQL** à l'aide de bibliothèques de NLP (spaCy, scikit-learn, RapidFuzz) — **sans aucune API de LLM**.

1 Partie de l'ontologie couverte

Élément	Contenu (ontologie/pharma.ttl)
Classes (3 niveaux)	Molecule → Antibiotique (Pénicilline, Macrolide) · Antalgique (non opioïde, opioïde) — Medicament → Princeps · Générique → …SurOrdonnance / …SansOrdonnance
Object properties	contientMolecule ↔ estContenuDans (inverses) · interagitAvec (symétrique) · peutProvoquer · aContreIndication · estIndiquePour · estContreIndiquePour (chaîne OWL)
Data properties	codeATC, doseMaxJourMg (molécule) · nomCommercial, dosage, formeGalenique, surOrdonnance, prixDT (médicament)
Axiomes exploités	classes définies (owl:equivalentClass) Princeps/Générique sur/sans ordonnance · ≥ 1 molécule par médicament · disjonctions
Individus	8 molécules (Paracétamol, Ibuprofène, Aspirine, Tramadol, Codéine, Amoxicilline, Azithromycine, Metformine) · 9 médicaments

2 Architecture et communication back-end ↔ front-end

Couche	Technologies	Rôle
Données	RDF/OWL (Turtle), owlrl	Ontologie chargée en mémoire ; le raisonneur OWL 2 RL écrit les faits déduits dans un **graphe nommé séparé** (déclaré vs inféré)
Back-end	Python, FastAPI, rdflib	API REST JSON (19 routes) ; **chaque réponse renvoie les requêtes SPARQL exécutées** ; paramètres échappés, identifiants validés
Front-end	React 18, Vite, React Router	Site vitrine + 6 pages ; appels via un client HTTP unique (proxy /api) ; états chargement / erreur / vide
IA	spaCy, scikit-learn, RapidFuzz	Modèle local PharmaNLU-A (253 Ko) exposé par POST /api/nlp/ask

Le CRUD écrit dans `backend/data/runtime.ttl` : l'ontologie commune de l'équipe n'est jamais modifiée (bouton « Réinitialiser »).

3 Requêtes SPARQL — 16 requêtes nommées, exécutables depuis la console

Requêtes	Notion SPARQL
Q01 / Q02	Hiérarchie rdfs:subClassOf+ ; appartenance indirecte rdf:type/rdfs:subClassOf*
Q03 / Q04	FILTER numérique + booléen, ORDER BY ; recherche REGEX insensible à la casse + UNION
Q05	Propriété inverse estContenuDans (déduite par le raisonneur)
Q06	Génériques de même composition : double FILTER NOT EXISTS + économie calculée (BIND)
Q07 / Q07b	Interactions (propriété symétrique) et doublons de principe actif, VALUES

Requêtes	Notion SPARQL
Q08	Chaîne contientMolecule/aContreIndication/concernePathologie (≡ owl:propertyChainAxiom)
Q09 / Q10	OPTIONAL + FILTER ; agrégats COUNT, AVG, MIN, MAX, GROUP BY, HAVING
Q11 / Q12 / Q13	Négation FILTER NOT EXISTS ; requête ASK ; requête CONSTRUCT (graphe affiché)
Q14	CRUD : INSERT DATA, DELETE/INSERT WHERE, DELETE WHERE
Q15	Types inférés non déclarés : comparaison de graphes nommés (GRAPH)

4 Interfaces graphiques

Page	Fonctionnalités
Molécules	Tableau filtrable (classe via la hiérarchie, code ATC, recherche avec ou sans accents), arbre à 3 niveaux, fiche détaillée, création / modification / suppression
Médicaments	Cartes filtrables (type, ordonnance, forme, curseur de prix), statistiques SPARQL, fiche avec contre-indications déduites et graphe interactif, CRUD
Comparateur · Interactions	Princeps ↔ génériques avec économie en DT et % · vérification d'une association de 2 à 10 médicaments
Assistant IA · Console	Questions en langage naturel · catalogue des 16 requêtes en un clic + éditeur (lecture seule), export CSV

Sur chaque page, un panneau « Requêtes SPARQL exécutées » montre exactement ce qui a été envoyé au serveur. Badges « inféré » sur les faits déduits.

5 Module IA — modèle « PharmaNLU-A » : question en français → requête SPARQL

Modèle **hybride** développé par nous : classifieur supervisé + règles linguistiques. Il n'utilise **aucun LLM** (ni OpenAI, ni Gemini) : tout est exécuté localement et le modèle entraîné est versionné dans le dépôt Git.

Étape	Bibliothèque	Traitement — exemple « Quels médicaments contiennent de l'ibuprophène ? »
1. Normalisation	spaCy 3.8 + fr_core_news_sm	minuscules, accents, tokenisation, lemmatisation (contiennent → contenir)
2. Entités	spaCy EntityRuler	dictionnaire **généré automatiquement depuis l'ontologie** (un médicament ajouté par le CRUD est reconnu sans ré-entraîner)
3. Fautes de frappe	RapidFuzz 3.14	« ibuprophene » → **Ibuprofène** (similarité 86 %)
4. Délexicalisation	—	les entités deviennent xmol / xmed / xpatho / xclasse : « quel medicament contenir de le xmol »
5. Intention	scikit-learn 1.9	TF-IDF mots (1–2-grammes) + caractères (3–5) → régression logistique : 12 intentions, confiance 97 %
6. Filtres	règles (regex)	prix (« moins de 5 dinars »), ordonnance, générique/princeps, forme, gravité, attribut demandé
7. SPARQL	gabarits + rdflib	gabarit de l'intention rempli avec l'URI :Ibuprofene → exécution → réponse en français + requête affichée

96,9 %	100 %	314	12
intentions correctes sur 96 questions de test jamais vues	entités reconnues sur le jeu de test	questions d'entraînement (gabarits écrits à la main)	intentions → 12 gabarits SPARQL

Sécurité : sous 40 % de confiance, aucune requête n'est générée (« je n'ai pas compris ») ; s'il manque une entité, l'assistant le dit précisément. La page « À propos du modèle » affiche les bibliothèques, la précision et le F1 par intention. Entraînement reproductible (graine 42) : `python -m nlp.train`.

6 Valeur ajoutée

Comparateur princeps / générique	Trouve les médicaments de composition identique et calcule l'économie : Advil 6,50 DT → Ibuprofène Mylan 3,90 DT (−40 %).
Vérificateur d'interactions	Détecte les interactions (déclarées dans un seul sens, déduites dans l'autre grâce à la symétrie OWL) et les doublons de principe actif.
IA explicable	Intention, confiance, entités corrigées, étapes du pipeline et requête SPARQL commentée affichées pour chaque question.
Inférence visible	Badges « inféré » : classes définies, propriétés inverses et symétriques, contre-indications déduites par la chaîne de propriétés.
Qualité	36 tests automatisés (API, requêtes, NLP), graphe CONSTRUCT interactif, export CSV, historique des questions, responsive mobile.

7 Scénario de démonstration (≈ 5 minutes)

1	Accueil (site vitrine) → « Ouvrir l'application » : l'API affiche le nombre de triplets déclarés et inférés.
2	Assistant : « quels médicaments contiennent de l'ibuprophene » (faute volontaire) → Advil, Ibuprofène Mylan ; intention, confiance, requête générée.
3	Comparateur : Advil → Ibuprofène Mylan, économie de 40 %.
4	Interactions : Advil + Aspégic → interaction ibuprofène ↔ aspirine ; + Ibuprofène Mylan → alerte doublon.
5	CRUD : création de « Paracétamol Biogaran » (générique, sans ordonnance) → classé automatiquement GeneriqueSansOrdonnance (inféré) et reconnu par l'assistant.
6	Console SPARQL : chaîne de propriétés (Q08), agrégats + HAVING (Q10), NOT EXISTS (Q11), types inférés (Q15) — puis « Réinitialiser les données ».

8 Installation et portabilité

Installer	`git clone` … puis `scripts\setup.bat` (Windows) ou `scripts/setup.sh`
Lancer	`scripts\start.bat` → site http://localhost:5173 · API documentée http://localhost:8000/docs
Garanties	Versions épinglées, modèle spaCy et modèle entraîné inclus dans le dépôt, aucune clé API, chemins relatifs ; testé sur un clone neuf (36 tests verts).